Most generally, setting the *type* of a variable, attribute, or method argument limits the set of values that can be assigned to that variable, attribute, or method argument. The type also determines the behavior of the variable within the program. For example, the result of `x / y` depends on the type of `x` and `y`. In many languages the result will be different if the numbers are integers than if they are floating point types.

Many programming languages have a set of "basic types" that can be used, often including strings, boolean values, different kinds of numerical values, and so on. Ruby does not have basic types; every variable in Ruby is an instance of a class, and that class determines the behavior of the variable. In Ruby, `x / y` is equivalent to the method call `x./(y)` and the behavior depends on what class `x` is.

In many typed programming languages, you must declare the type of a variable before it is used, this is called *explicit typing*. Other programming languages can infer the type of a variable from its first use, so if I say `let x = 3` in TypeScript, TypeScript knows that `x` is a number. This is called "type inference."

In either case, there is usually a tool, often part of the compiler, that evaluates every variable interaction to see if type information is followed. If, later in the TypeScript code, I try to say `x = "foo"`, TypeScript will give a compilation error because `"foo"` is a string. This is called *static typing*.

Ruby, without type information, does not do this. In Ruby, the type of a variable is determined while the code is running by the variables that are assigned to it, and Ruby determines if the variable can receive a method only at runtime. This is called *dynamic typing*, and the process of determining the behavior of the method at the last possible moment is called *late binding*.

There's another distinction here that isn't as useful to us, where in some languages the type barriers are more permeable, and if you type `3 + "3"`, the language will automatically coerce the string to an integer and allow the addition to continue. This is called *weak typing*, and languages that don't do this have *strong typing*. You will sometimes see "strong typing" incorrectly used as a synonym for "static typing," but there are two different concepts and it's useful to keep them separate.

There are benefits to static typing and a compilation step that validates all assignments:

- If the compiler and runtime know information about what type to expect, they can often optimize internal behavior and improve performance.
- A person reading the code can get more information about the intent and behavior of the code if there is type information.
- A developer tool like an IDE or editor can use type information to provide information to the developer as the code is being written.

There are also some drawbacks:

- Statically typed code is usually more verbose than dynamic code. Though type inferencing has improved this situation.
- Sometimes a developer has to spend time convincing the type system that the code that has been written is correct.
- Statically typed code is often less flexible than dynamic code and harder to change. (To be fair, lots of people would see this as an advantage.)